



-
- | | |
|--|---|
| <p>1. Cozile memoreaza ambele capete ale structurii, stivele doar un capat.
J: O coada este o structura liniara de tip lista care restrictioneaza adaugarile la un capat si stergerile la celalalt capat, iar stiva este o structura liniara de tip lista care restrictioneaza adaugarile si stergerile la un singur capat</p> | <p>Una dintre diferențele dintre coadă și stivă este: Justificați
a) cozile necesită liste înlănțuite, iar stivele nu b) stivele necesită liste înlănțuite, iar cozile nu c) cozile memorează ambele capete ale structurii, stivele doar un capăt d) stivele memorează ambele capete ale structurii, cozile doar un capăt</p> |
| <hr/> | |
| <p>2. -Cea care a fost inserata prima.
J : Se aplica principiul de FIFO</p> | <p>Pp. ca se apeleaza operatia de accesare element intr o Coad-aCuPrioritati avand exact 2 elemente cu prioritate egala. Cum se va selecta valoarea returnata?</p> |
| <hr/> | |
| <p>3. J: Adancimea minima a lui T este atunci cand T este aproape plin</p> | <p>Pp. ca T este un arbore binar cu 14 noduri , care este adancimea minima a lui T?</p> |
| <hr/> | |
| <p>4. - oricate
J: Daca tabela este plina (prim-liber = m) => redimensionare: marim m, ceea ce presupune redispersarea elementelor (re-hashing).</p> | <p>O TD cu coliziuni rezolvate prin adresare deschisa si verificare patratica are 1024 locasii. Care este nr. maxim de intrari care pot fi plasate in tabela?</p> |
| <hr/> | |
| <p>5. - Cand nr. de prioritati posibile este foarte mare
J: Deoarece necesita un vector urias si ineficient dpdv al memoriei.</p> | <p>Un vector de cozi se poate folosi pt. a implementa o Coad-aCuPrioritati, in care fiecărei prioritati ii corespunde un element din vector. Cand aceasta implementare nu este fezabila?</p> |
-



-
6. - $h+1$ Pp. ca T este un arbore binar cu inaltimea h. Care este nr. minim de noduri a lui T?
J: Daca fiecare nod ar a avea un singur fiu atunci am avea nr. minim de noduri
-
7. - fals Algoritmul de sortare HeapSort are proprietatea de sortare stabila?
J: Operatiile din ansamblu pot schimba ordinea elementelor egale
-
8. - adaugare Pp. o Colectie implementata folosind o LSI. Care din operatiile de mai jos au complexitate Theta(1)
J: Deoarece presupune doar alocarea unui nou nod si legarea lui de restul listei, fara a parcurge elemente
-
9. - Orice arbore binar are cel putin un nod Selectatia afirmatiia falsa:
- Orice nod dintr-un arbore are cel mult doi fi
-
10. - Orice arbore binar plin este si Alegeti afirmatia corecta:
complet
J: Plin = fiecare nod intern are 2 copii
Complet = Toate nivelurile sunt pline mai putin ultimul (insa fara gauri, nodurile sunt de la stanga la dreapta fara gap-uri)
-
11. Se considera un vector de nr. reale. Alegeti algoritmi de sortare care pot fi folositi pt. a ordona vectorul



- Toate in afara de RadixSort
J: RadixSort merge doar pe numere intregi, nu nr. reale

12. - sa parcurga elementele unui container unul cate unul
J: Deoarece iteratorul are metode precum `prim`, `urmator`, `element`, ce faciliteaza parcurgerea containerului element cu element.

13. - timp liniar
J: deoarece in cel mai rau caz (daca elementul nu se afla in vector) va trebui sa parcurgem fiecare element rand pe rand

Care este scopul principal al unui iterator?
Care este Cazul Defavorabil pt. cautarea secventiala intr-un vector?

14. - m/s
J: Factorul de incarcare al TD este raportul dintre nr. de valori si nr. de locatii, si reprezinta nr. mediu de valori memorate la o locatie.

Pp. ca se adauga M valori intr-o TD cu S locatii. Care este formula corecta pentru factorul de incarcare al tablei?

15. - $\log_2 n$
J: in cazul in care arborele binar asociat ansamblului ar fi plin, iar h este inaltimea arborelui, pe fiecare nivel am avea 2^{nivel} noduri
nr noduri (n) = $2^0 + 2^1 + \dots + 2^h$

Care este formula cea mai buna pt. aproximarea inaltimei unui ansamblu cu n noduri?



=> $h = \log_2(n+1)-1$ apartine
 $\theta(\log_2 n)$

16. - $O(n)$

J: Ștergerea unui element dintr-un vector ordonat necesită deplasarea tuturor elementelor de după el cu o poziție spre stânga, ceea ce în cel mai rău caz implică $O(n)$ operații.

CF: el. se afla in relatie cu x_1
=> nu este in vector $T(n) = \theta(1)$

CD: Elementul se afla pe pozitia p si trebuie sters => $T(n) = p$ (pt a gasi elementul) + $n-p$ (pt a muta elementele) => $T(n) = n$ apartine $\theta(n)$

C. Ștergerea unui element e dintr-un vector ordonat se poate face în:

a) $O(\log_2 n)$ b) $O(n)$ c) $\theta(n)$ d) $\theta(\log_2 n)$

Justificati

17. - amândouă se modifică

J: La adăugarea unui element într-o coadă vidă, noul nod devine și primul, și ultimul element, deci ambele referințe trebuie actualizate.

C. Presupunem o implementare înlănțuită a unei Cozi, memorând în 2 variabile referințe la primul, respectiv ultimul nod al cozii.

Care dintre aceste două variabile se modifică la o adăugare într-o coadă VIDĂ? Justificati

a) niciuna b) doar primul se modifică c) doar ultimul se modifică d) amandoua se modifică

18. - fals

Justificare: desenezi contraexemplu

Inserarea într-un ABC este "comutativă". Adică, inserându-l pe x și apoi pe y în arbore este echivalent cu inserarea lui y și apoi a

lui x . Justificati a) adevărat b) fals



19. - $O(n \log_2 n)$
J: În cazul defavorabil, Heap-Sort efectuează n extrageri din heap, fiecare având complexitate $O(\log_2 n)$ pentru reechilibrare, rezultând un timp total de $O(n \log_2 n)$
 .Care este timpul defavorabil pentru sortarea unui vector cu n elemente folosind HeapSort? Justificati
 a) $O(\log_2 n)$ b) $O(n)$ c) $O(n \log_2 n)$ d) $O(n^2)$
-
20. - b) atât adăugarea, cât și ștergerea se fac în timp liniar
J: Dacă vârful stivei este la începutul vectorului ($v[0]$), iar noile elemente sunt adăugate spre final, fiecare push și pop necesită deplasarea tuturor elementelor din vector pentru a menține vârful la $v[0]$, ceea ce duce la timp liniar $O(n)$ pentru fiecare operație.
 Presupunem o Stivă implementată secvențial pe un vector v . Ce se întâmplă în cazul în care memorăm vârful stivei la locația $v[0]$, iar primul element introdus în stivă la ultima poziție utilizată în vector? Justificati
 a) atât accesul, cât și ștergerea se fac în timp liniar b) atât adăugarea, cât și ștergerea se fac în timp liniar
 c) stiva nu poate fi folosită pentru a evalua expresii postfixate
-
21. e) niciuna
J: vida: verificam dacă prim este NIL
 accesare: informația utilă din pointerul prim
 ștergere: prim devine $[\text{prim}].\text{urm}$
 adăugare: se creează un nod nou și se face legătura cu prim și se actualizează prim
 În implementarea Stivei folosind o listă înlanțuită, ce operații necesită timp liniar în cazul defavorabil? Justificati
 a) vidă b) accesare (operația element) c) ștergere d) adăugare
 e) niciuna din operațiile menționate



a) data[0]

J: Intr-un ansamblu cu rel $\geq$, fiecare el este mai mare sau egal cu fii lui. Drept urmare valoarea cea mai mare va fi data[0], stocata in radacina

Dacă un ansamblu este implementat folosind un vector numit data, și acest vector conține n elemente ($n > 0$), unde este valoarea

cea mai mare într-un ansamblu construit cu relația $\geq$ Justificati
a) data[0] b) data[n-1] c) data[n] d) data[2*n + 1] e) data[2*n + 2]

23.

C. Pentru adăugarea unei chei într-o TD în care coliziunile se rezolvă prin înlănțuire (liste independente), timpul de execuție în cel mai rău caz este proporțional cu factorul de încărcare al tabelului
a) adevărat b) fals

24. a) lista implementata secven-

**tial
J: La lista implementata secvential avem acces la elemente de pe o anumita pozitie in timp constant, pe cand la listele inlantuite operatia se face in timp liniar**

C. Care tip de listă este cea mai potrivită pentru a răspunde la întrebarea "Care este elementul de pe poziția n?" Justificați
a) lista implementată secvențial b) lista dublu înlănțuită c) lista simplu înlănțuită d) listele simplu și dublu înlănțuite